🌿 STOCKY AI

Proactive AI Business Partner for Malaysian Produce Wholesalers

---

PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026 | Version 1.0

umhackathon@um.edu.my

Powered by Z.ai | ilmu-glm-5.1 | YTL AI Labs

Team Name	Grabbeez
Repository	https://github.com/Vaarshh17/StockyAI
Domain	AI for Economic Empowerment & Decision Intelligence
Document Version	1.0
Date	24 April 2026

Table of Contents

Table of Contents 2
1. Project Overview 4
Problem Statement 4
Proposed Solution Summary 4
2. Background & Business Objective 5
2.1 Background of the Problem 5
2.2 Importance of Solving This Issue 5
2.3 Strategic Fit / Impact 5
3. Product Purpose 6
3.1 Main Goal of the System 6
3.2 Intended Users (Target Audience) 6
4. System Functionalities 7
4.1 Description 7
4.2 Key Functionalities 7
Feature 1: Proactive Morning Brief (Flagship) 7
Feature 3: Photo OCR → Auto Inventory 7
Feature 4: Festival Autonomous Alert 7
Feature 5: Forwarded Message Intelligence 7
Feature 6: Voice Note Support 8
Feature 7: Stocky's Instinct 8
Feature 8: Spoilage & Velocity Monitoring 8
Feature 9: Credit & Receivables Tracking 8
4.3 AI Model & Prompt Design 8
4.3.1 Model Selection: ilmu-glm-5.1 (Z.ai / YTL AI Labs) 8
4.3.3 Context & Input Handling 9
4.3.4 Fallback & Failure Behaviour 9
5. User Stories & Use Cases 11
5.1 User Stories 11
5.2 Use Cases (Main Interactions) 11
UC-01: Morning Brief → Approval → Execution 11
UC-02: Photo Delivery Note → Auto-Log 11
UC-03: Forward Price Quote → Cross-Reference Analysis 11
UC-04: Festival Prep Alert (Autonomous, No Trigger) 12
6. Features Included (Scope Definition) 13
7. Features Not Included (Scope Control) 14
8. Assumptions & Constraints 15
8.1 LLM Cost Constraints 15
8.2 Technical Constraints 15
8.3 Performance Constraints 16
8.4 User Input Assumptions 16

1. Project Overview

Problem Statement

Malaysian wet market wholesalers, the backbone of the national food supply chain, operate with zero digital infrastructure. A typical trader at Pasar Borong Selayang manages 8–15 perishable commodities, 5–10 supplier relationships, and credit extended to 20–50 buyers, all tracked mentally or on paper.

Every morning, a correct buying decision requires cross-referencing four independent signals simultaneously:

- Current inventory levels and expiry dates per commodity
- Supplier price quotes vs. FAMA government benchmark
- 3–5 day weather forecast (rain = slow market sales = accelerated spoilage)
- Upcoming Malaysian festivals that cause demand spikes 7–14 days ahead

No existing tool does this. The trader's instinct (built over decades) is the only processing system. When that instinct is wrong, spoilage losses of RM500–RM5,000 per batch are common. Credit defaults go untracked. Festival buying windows are missed.

Proposed Solution Summary

Stocky AI is a proactive AI agent — not a chatbot — delivered via Telegram. It does not wait to be asked. Every morning at 3:30 AM, before the trader wakes up, Stocky has already analysed all four signals, generated a specific buying plan, and queued it for one-tap approval. Throughout the day, it monitors spoilage risk, sell-through velocity, and overdue credit autonomously — firing alerts only when thresholds are crossed.

The core differentiator: Stocky executes, not just advises. When the trader approves a morning brief, Stocky logs the inventory buy and generates pre-written WhatsApp order messages — in Malay or Mandarin — to each supplier, ready to send in one tap.

2. Background & Business Objective

2.1 Background of the Problem

Malaysia's wet market supply chain moves approximately RM12 billion in fresh produce annually. Pasar Borong Kuala Lumpur alone processes over 3,000 tonnes of vegetables and fruits per day. Yet the traders at the core of this chain — the wholesalers who sit between farmers/importers and retailers — operate almost entirely without digital tools.

The existing workflow is: receive supplier WhatsApp price quotes in the morning, mentally compare against what they know about FAMA prices, decide what to buy, record the delivery on paper, track credit in a physical ledger, and try to remember which stock is aging. This system fails in predictable ways:

- Spoilage losses: Perishables (bayam, kangkung, tomato) have 3–7 day shelf lives. A rainy week that slows market foot traffic can wipe out an entire batch.
- Missed festival demand: Hari Raya, Chinese New Year, and Deepavali each create 2–3× demand spikes for specific commodities. Traders who buy 7–14 days early capture margin; those who wait face both stockouts and inflated supplier prices.
- Credit defaults: Extended credit to retail buyers (hutang) is standard practice. Without a tracking system, follow-up is inconsistent and cash flow suffers.
- Price opacity: FAMA publishes weekly benchmark prices but traders have no easy way to compare supplier quotes against this benchmark in real time.

2.2 Importance of Solving This Issue

The downstream impact of poor wholesale decisions propagates through the entire food supply chain. Retailers who cannot source reliable supply raise prices. Consumers pay more for essential goods. By improving decision quality at the wholesale layer, Stocky AI contributes to price stability in the national food supply.

At the individual trader level: a 10% reduction in spoilage losses on a RM50,000/month operation saves RM5,000/month. Capturing one additional festival buying window per quarter adds meaningful margin. Recovering even 50% of currently untracked credit improves cash flow materially.

2.3 Strategic Fit / Impact

Stocky AI is built natively on Z.ai's ilmu-glm-5.1 model (ILMU API, YTL AI Labs) — a model trained with Malaysian language and cultural context. This is not incidental: the traders Stocky serves communicate in Bahasa Malaysia, Mandarin, and code-switched Malay-English. A generic English-first LLM would fail at the first interaction.

The product also aligns with Malaysia's national digitalisation agenda (MyDIGITAL) and the government's push to modernise the agricultural supply chain under the National Agrofood Policy 2.0. A Telegram-native delivery removes the app download barrier entirely — 93% of Malaysian smartphone users already have Telegram installed.

3. Product Purpose

3.1 Main Goal of the System

To be the first AI agent that thinks, decides, and acts on behalf of Malaysian produce wholesalers — autonomously monitoring their business signals, surfacing cross-signal insights they would otherwise miss, and executing approved actions without friction.

The product's north star metric is not engagement — it is decisions improved per day. Stocky is successful when a trader makes a better buying decision at 4 AM because Stocky already ran the analysis, or when spoilage is avoided because Stocky fired a sell-now alert 48 hours before the batch expired.

3.2 Intended Users (Target Audience)

User Type	Profile	Primary Need
Primary: Fruit & Vegetable Wholesaler	Age 35–60, operates at Pasar Borong, manages 5–15 commodities, communicates in Malay/Mandarin	Proactive morning brief, spoilage alerts, buy execution
Secondary: Small Produce Importer	Imports from Thailand/China, manages supplier relationships across language barriers	Multilingual supplier messaging, price comparison vs FAMA
Tertiary: Wet Market Retailer (buyer)	Buys on credit from the wholesaler, less tech-savvy	Indirectly benefits: more reliable supply, faster credit settlement

4. System Functionalities

4.1 Description

Stocky AI operates as an autonomous multi-signal intelligence engine delivered through Telegram. The system continuously monitors seven data streams — inventory levels, supplier price history, FAMA government benchmarks, sell-through velocity, weather forecasts, credit receivables, and Malaysian festival calendar — and fires actions when pre-defined thresholds are crossed, without waiting for user input.

The architecture follows a three-layer model: (1) Data Layer (SQLAlchemy async ORM over Supabase PostgreSQL), (2) Agent Layer (ilmu-glm-5.1 tool-calling loop via ILMU API), and (3) Action Layer (Telegram bot with approval keyboards + WhatsApp deep link generation).

4.2 Key Functionalities

Feature 1: Proactive Morning Brief (Flagship)

Fires autonomously at 3:30 AM daily. The agent loop calls four tools in sequence — get_inventory, get_weather_forecast, compare_supplier_prices, get_outstanding_credit — synthesises all signals, and generates a structured buying plan with specific quantities, suppliers, and prices. Proposed orders are presented with a one-tap Approve / Skip keyboard

.

Feature 2: Approval → Execution Loop

When the trader taps Approve, Stocky does not just acknowledge — it executes: logs each buy to the inventory database, then generates pre-written WhatsApp order messages in the supplier's preferred language (Malay or Mandarin) as wa.me deep links. One tap per supplier sends the order. Zero re-typing.

Feature 3: Photo OCR → Auto Inventory

Trader photographs a delivery note or invoice. EasyOCR (local, no API cost) extracts text. ilmu-glm-5.1 parses commodity name, quantity, price, and supplier from the unstructured text. update_inventory is called automatically. No manual data entry required.

Feature 4: Festival Autonomous Alert

Checks daily whether a major Malaysian festival is 7–14 days away (the optimal buying window before prices spike). If stock for a relevant commodity will run out before the festival at current sell rate, Stocky fires a pre-emptive buying alert with recommended quantity and cheapest supplier — unprompted.

Feature 5: Forwarded Message Intelligence

Trader forwards a supplier WhatsApp price quote directly to Stocky. Before responding, the agent is forced (via system prompt injection) to call get_inventory and compare_supplier_prices to cross-reference the quote against current stock and FAMA benchmark. Response includes: accept/reject recommendation with data.

Feature 6: Voice Note Support

Trader sends a voice note in Malay (e.g. 'Jual 200kg tomato kepada Pasar Borong, harga 3.20 sekilogram'). faster-whisper transcribes locally. ilmu-glm-5.1 interprets and calls the appropriate tool (log_sell, log_credit, update_inventory). Transcription echoed back for confirmation.

Feature 7: Stocky's Instinct

Appended to every morning brief and weekly digest. A second Z.ai call analyses 14 days of cross-signal data — velocity trends, supplier price cycles, credit accumulation patterns, inventory timing — and surfaces ONE non-obvious pattern the trader likely hasn't noticed. Not a summary; a genuine inference.

Feature 8: Spoilage & Velocity Monitoring

Spoilage check fires at 8 AM and 2 PM: alerts only when an item has ≤2 days remaining AND quantity >100kg AND rain is forecast. Velocity alert fires every 4 hours: alerts only when sell rate deviates >30% from 7-day baseline. No-alert silence is intentional — prevents notification fatigue.

Feature 9: Credit & Receivables Tracking

Log credit sales, track due dates, mark as paid. 9 AM daily reminder fires only for amounts >RM50 due within 24 hours. Stocky drafts a credit reminder message in the buyer's language for approval before sending.

4.3 AI Model & Prompt Design

4.3.1 Model Selection: ilmu-glm-5.1 (Z.ai / YTL AI Labs)

ilmu-glm-5.1 is accessed via the ILMU API (api.ilmu.ai/v1), which exposes an endpoint that is OpenAI-compatible. The model was selected for four reasons specific to this use case:

- The target users communicate in Bahasa Malaysia and Mandarin, often code-switched. ilmu-glm-5.1 was developed with Malaysian linguistic context — generic English-first models (GPT-4, Claude) handle Malay poorly at colloquial register.
- The model natively supports OpenAI-format function/tool calling, which is the core mechanism of Stocky's agent loop. Without reliable tool-calling, the agentic architecture collapses.
- The morning brief requires synthesising outputs from 4 sequential tool calls into a coherent, prioritised buying plan. ilmu-glm-5.1 maintains context across the full tool chain.
- As a locally-developed Malaysian model, inference costs via ILMU API are substantially lower than GPT-4 or Claude API, which is critical for always-on scheduler jobs that fire 10+ times per day.

4.3.2 Prompting Strategy: Multi-Step Agentic Prompting with Mandatory Tool Chains

Stocky uses multi-step agentic prompting — the model is placed in a loop where it autonomously decides which tools to call, in what order, based on the current context. The loop runs up to 6 iterations (MAX_TOOL_ITERATIONS = 6).

The system prompt encodes MANDATORY TOOL CHAINS — explicit sequences the model must follow for each scenario. This is a form of constrained few-shot prompting at the instruction level:

- **Morning Brief chain:** get_inventory → get_weather_forecast → get_outstanding_credit → compare_supplier_prices → synthesise
- **Buy recommendation chain:** get_inventory → get_weather_forecast → compare_supplier_prices → recommend
- **Spoilage check chain:** get_inventory → get_weather_forecast → assess_risk

This strategy was chosen over zero-shot or pure ReAct because: (a) the domain has well-defined decision sequences that should not vary, (b) it prevents the model from responding based on training knowledge alone without checking live data, and (c) it produces consistent, auditable output formats that traders can rely on.

For proactive scheduler jobs (no user input), the prompt is injected programmatically — the user never sees it. For forwarded messages, an INSTRUCTION block is prepended to the user message: 'Before replying, you MUST call the relevant tools to check live data. Do NOT respond based on the forwarded text alone.'

4.3.3 Context & Input Handling

Each user maintains a conversation history stored in memory (in-memory dict, keyed by Telegram user_id). History is trimmed to the last 10 turns to prevent context overflow. Proactive scheduler prompts are explicitly excluded from history (save_history=False) to prevent synthetic 'user' messages from polluting the real conversation.

User persona (name, language, commodities, city) is stored in Supabase and injected at the top of every system prompt as a USER PROFILE block that overrides all language and commodity defaults. Maximum input size per message is governed by the API's context window; inputs are not chunked — oversized inputs (e.g. very long OCR text) are passed in full as the model needs the complete delivery note to extract all items.

For Photo OCR: image bytes are processed by EasyOCR locally (no API call), producing a text string that is then passed to the model. This keeps vision costs at zero and avoids dependency on a vision-capable API endpoint.

4.3.4 Fallback & Failure Behaviour

The following failure modes are handled explicitly:

- **Empty model response:** If the final loop iteration produces no text content (e.g. model returned tool_calls only and was cut off), a Malay-language fallback is returned: 'Maaf, saya menghadapi masalah teknikal. Sila cuba lagi.
- **API timeout / rate limit:** call_llm() includes a 2-attempt retry with asyncio.sleep(2) between attempts. If both fail, the exception propagates to the scheduler job which logs the error and stays \silent (no broken message sent to user).

- **Tool execution failure:** execute_tool() wraps all tool calls in try/except. On failure, it returns {"error": str(e), "tool": name} which the model receives as a tool result — it can then respond gracefully rather than crashing.
- **Hallucinated tool arguments:** The dispatcher filters args to known parameter names only before calling each function. If the model passes unexpected fields, they are silently dropped rather than causing TypeErrors.
- **Instinct failure:** get_instinct() is wrapped in a silent exception handler — if the 14-day analysis call fails for any reason, the morning brief is sent without the Instinct suffix. It is additive, not critical.
- **Forwarded message duplicated:** MD5 hash + 60-second window prevents duplicate responses if the same message is forwarded twice in quick succession.

5. User Stories & Use Cases

5.1 User Stories

#	As a...	I want to...	So that...
US-01	Wholesaler, I wake up at 4 AM before heading to the market	Receive a complete buying plan already prepared — what to buy, from whom, at what price	I can make buying decisions before suppliers' stock runs out
US-02	Wholesaler with 8 commodity lines	Approve all proposed orders in one tap	WhatsApp order messages are sent to each supplier without me typing anything
US-03	Wholesaler receiving a delivery	Take a photo of the delivery note	Stock is logged automatically without manual entry
US-04	Wholesaler who received a supplier price quote on WhatsApp	Forward the message to Stocky	Know immediately if the price is good vs FAMA and vs my current stock need
US-05	Wholesaler 10 days before Hari Raya	Be alerted that tomato and cili demand will spike	I can bulk-buy before prices rise and avoid stockouts during the festival
US-06	Wholesaler at the market (hands busy)	Send a voice note: 'Jual 300kg bayam RM1.80'	The sale is logged and stock updated without stopping to type
US-07	Wholesaler with overdue credit from a buyer	Get a morning alert when payment is due today	I can follow up before the buyer has moved on to the next transaction
US-08	Wholesaler at end of Monday	Receive a weekly business digest with revenue by commodity	I can see which commodities are profitable and which are dragging performance

5.2 Use Cases (Main Interactions)

UC-01: Morning Brief → Approval → Execution

Trigger: Scheduler fires at 3:30 AM. Agent calls 4 tools. Brief generated with proposed orders. Trader taps. Inventory updated. WhatsApp links generated. Trader taps each link to send order. Done.

UC-02: Photo Delivery Note → Auto-Log

Trigger: Trader photographs delivery note. EasyOCR extracts text. Agent parses and calls update_inventory for each line item. Trader receives confirmation. No form-filling.

UC-03: Forward Price Quote → Cross-Reference Analysis

Trigger: Supplier sends price quote on WhatsApp. Trader forwards to Stocky. Agent forced to call compare_supplier_prices and get_inventory before responding. Returns: accept/negotiate/reject recommendation with data justification.

UC-04: Festival Prep Alert (Autonomous, No Trigger)

Trigger: festival_prep_job checks daily at 10 AM. Conditions: festival within 7–14 days + relevant commodity runs out before festival at current velocity. If met: alert fired with buying recommendation and approval keyboard. If not: silence.

6. Features Included (Scope Definition)

- Proactive morning brief with multi-signal analysis (inventory + weather + prices + credit) delivered at 3:30 AM daily without user prompt
- Approval → execution loop: approved orders logged to database + WhatsApp deep links generated per supplier in Malay or Mandarin
- Photo OCR pipeline: delivery note photo → EasyOCR → Z.ai parsing → auto inventory update
- Festival autonomous alert: pre-emptive buying recommendation when festival is 7–14 days away and stock will run short
- Forwarded message intelligence: forces tool cross-reference before responding to forwarded supplier price quotes
- Voice note support: faster-whisper local transcription → Z.ai interpretation → tool execution
- Stocky's Instinct: 14-day cross-signal pattern analysis appended to morning brief and weekly digest
- Spoilage monitoring (8 AM + 2 PM) with rain-forecast integration — silent when no risk
- Sell-through velocity alerts every 4 hours — fires only on >30% deviation from baseline
- Credit receivables tracking with daily 9 AM reminder for amounts due today
- Weekly Monday digest with revenue by commodity, best/worst performers, outstanding credit
- FAMA government benchmark price integration (weekly data) for supplier quote evaluation
- 4-question onboarding: name, language, commodities, city — persona persisted to Supabase
- Trilingual support: Bahasa Malaysia, English, Mandarin (model detects and matches user language)
- Conversation history per user (last 10 turns) for contextual responses
- Supabase PostgreSQL as primary database with SQLite fallback for local development
- 6 APScheduler cron jobs: morning_brief, spoilage_check, velocity_alert, credit_reminder, monday_digest, festival_prep

7. Features Not Included (Scope Control)

- Direct bank account or payment gateway integration — Stocky tracks credit records but does not initiate financial transactions
- Real-time FAMA price scraping — FAMA benchmarks are seeded from weekly published data, not live-scraped (FAMA does not offer a public API)
- Supplier onboarding portal — suppliers are added to the database via trader interaction, not via a self-service flow
- Multi-business / multi-user account management — the current architecture is single-trader per Telegram account
- Native mobile app (iOS / Android) — delivery is exclusively via Telegram bot
- Production WhatsApp message sending — Stocky generates wa.me deep links for one-tap sending; it does not send WhatsApp messages autonomously (WhatsApp Business API requires approval and is out of scope)
- Dashboard web application — a React dashboard was designed but is not part of the core hackathon submission
- Direct supplier API integrations — supplier prices are entered via forwarded messages or manual input, not via API
- Image generation or visual reporting — all output is text-based, formatted for Telegram mobile display
- Multi-language within a single session — language is set at onboarding and is consistent throughout; mid-session switching is not supported

8. Assumptions & Constraints

8.1 LLM Cost Constraints

Token cost analysis per average user session:

Interaction Type	Avg Input Tokens	Avg Output Tokens	Frequency
Morning Brief (4 tool calls)	~2,400	~800	1× daily
Action Extraction (2nd call)	~600	~200	1× daily
User conversational query	~800	~300	~5× daily
Scheduler jobs (spoilage, velocity, credit)	~600	~200	~6× daily
Instinct analysis	~1,200	~150	1× daily

Estimated total: ~18,000 input tokens + ~5,000 output tokens per active user per day. At ILMU API rates, this is substantially below equivalent GPT-4 costs.

Cost management decisions made specifically to keep inference costs manageable:

- **Silence rules:** Scheduler jobs (spoilage, velocity, credit) are designed to stay silent when conditions are not met. Most jobs fire no LLM call on a typical day — they check data conditions first in Python before calling the model.
- **Fast model routing:** Simple scheduler jobs (spoilage, velocity, credit) are routed to MODEL_FAST (ilmu-glm-5.1, same endpoint, lower inference cost configuration). Complex reasoning (morning brief, instinct, digest) uses MODEL_SMART.
- **No history for proactive calls:** Scheduler-triggered agent runs pass save_history=False, preventing the accumulation of synthetic conversation history and keeping context windows lean.
- **Local OCR and transcription:** EasyOCR and faster-whisper run locally — zero API cost for photo and voice processing. Only the final interpretation step (structured data extraction) calls the LLM.

8.2 Technical Constraints

- Telegram polling: The bot uses long-polling (not webhooks) for the hackathon. Webhooks require a public HTTPS endpoint; polling works on local machines and cloud VMs without SSL configuration.
- APScheduler timezone: All scheduler jobs run in Asia/Kuala_Lumpur timezone. The morning brief at 3:30 AM is calibrated to fire before the 4 AM wholesale market opening.
- EasyOCR first-run latency: First EasyOCR call downloads ~100MB of model weights. Subsequent calls use the cached model. This must be pre-warmed before the demo.
- Supabase connection pooling: Async SQLAlchemy is configured with pool_size=5, max_overflow=10 for Supabase. SQLite is used for local development (no connection pooling needed).
- FAMA data: FAMA publishes weekly benchmark prices in PDF format — no machine-readable API exists. Prices are seeded from historical data into the database at init time.

8.3 Performance Constraints

- Morning brief latency: End-to-end from scheduler trigger to Telegram message delivery is typically 8–15 seconds (4 sequential tool calls + LLM synthesis). Acceptable for a pre-dawn message.
- OCR latency: After warm-up, EasyOCR processes a typical delivery note image in 2–4 seconds. This is shown to the user via a 'Membaca gambar...' loading indicator.
- Concurrent users: For the hackathon demo, the bot supports a single active user (ACTIVE_USERS set). Multi-user support requires per-user scheduler scoping which is architecturally straightforward but out of scope.

8.4 User Input Assumptions

- Traders are assumed to have a Telegram account and a smartphone with a camera.
- Voice notes are assumed to be in Malay or English — faster-whisper supports both.
- Delivery notes photographed are assumed to be legible (printed or clearly handwritten). Blurry or damaged notes fall back to manual entry.
- The model cannot act on business data it has not been given — if no inventory has been logged, the morning brief will correctly state no data is available rather than hallucinating stock levels.

9. Risks & Questions Throughout Development

Risk	Description	Impact	Mitigation
LLM Hallucination on Financial Data	Model invents stock levels, prices, or credit amounts not in the database	High — trader makes buying decision on false data	Mandatory tool chains in system prompt. NEVER answer factual questions without calling a tool first. Tested explicitly in unit tests.
OCR Failure on Handwritten Notes	Delivery notes handwritten in unclear Malay/Chinese characters fail EasyOCR extraction	Medium — trader must enter manually	Graceful fallback: 'Cuba ambil gambar lebih jelas, atau taip butiran secara manual.' Voice note is alternative input.
Festival Date Accuracy	Malaysian festival dates change annually (esp. Islamic calendar). Hardcoded 2025-2026 dates will be wrong in 2027.	Medium — festival alerts fire at wrong time	festivals.py is isolated and easily replaceable. Production version would pull from a public Malaysian public holiday API.
Weather API Availability	Open-Meteo free tier has rate limits and occasional downtime	Low — alerts proceed without weather context	System prompt instructs model to proceed without weather context if unavailable. Cached last-known forecast used where possible.
Duplicate Telegram Instances	Running two instances of main.py with the same BOT_TOKEN causes 409 Conflict	Low — development inconvenience only	Documented in README. run.bat script kills previous Python processes before starting. drop_pending_updates=True on polling start.
ILMU API Key Expiry	API key expires mid-demo or rate limit hit during live demonstration	High during demo — no responses	DEMO_MODE=True fallback returns mock responses. Screenshots of real output prepared as backup.
Trader Adoption Barrier	Target users (age 40–60, non-tech-savvy) may resist a new tool even via familiar Telegram interface	High for commercial deployment	4-question onboarding takes <2 minutes. No app download. Bot speaks in trader's language. First morning brief is the 'aha moment'.